



10강 실습 ·

파이썬으로 판단하기: 클라우드 vs 서버

누적 비용 곡선 · 민감도 분석 · 몬테카를로 — 확률로 말하는 의사결정

오늘 만들 것 — 세 개의 답



누적 비용 곡선

for문과 리스트로 월별 누적 비용을 계산하고, 두 곡선의 교차점(24개월)을 눈으로 확인한다.



민감도 분석

계산을 함수로 묶고 시나리오를 반복문으로 돌려, 가정이 흔들리면 답이 어떻게 변하는지 본다.



몬테카를로 시뮬레이션

9장의 random 모듈로 불확실한 미래를 10,000번 살아 보고 '구매가 이득일 확률'을 구한다.

오늘의 최종 산출물 "3년 기준, 서버 구매가 이득일 확률은 약 72%입니다." — 이 한 문장을 직접 계산해 냅니다.

시작 전 점검 —



환경 준비

- colab.research.google.com 접속 → 새 노트북
- 설치할 것 없음 — random, matplotlib 기본 제공
- 주피터 노트북 사용자도 동일하게 진행 가능



9강 기억 소환 (체크!)

- for문으로 리스트에 값 쌓기 — append
- random.gauss(평균, 표준편차)
- 몬테카를로 3단계: 흉내 → 반복 → 평균



오늘의 규칙: 코드는 눈이 아니라 손으로 배운다 — 모든 코드는 직접 타이핑합니다!

오늘의 문제: 클라우드 vs 서버

항목	서버 구매	클라우드
초기 비용	1,200만 원	0원
월 비용	20만 원 (관리·전기)	0.7만 원 × 사용 시간
현재 사용량	—	월 100시간 → 70만 원

이론 시간의 답

손익분기 = $1,200 \div (70 - 20) = 24$ 개월

오늘 코드로 답할 세 가지

Q1

그 '24개월', 그래프로 보이는가?

Q2

가정(사용량·요금)이 흔들리면 답은 어떻게 변하는가?

Q3

미래가 불확실하다면, 확률로는 뭐라고 말해야 하는가?

PART 1

25분

누적 비용 곡선

for문과 리스트만으로 교차점을 찾고, 그래프로 확인한다

- 1 문제의 숫자를 변수로 옮기기
- 2 for문으로 월별 누적 비용 쌓기
- 3 if문으로 교차점(역전의 순간) 찾기
- 4 matplotlib으로 두 곡선 그리기

문제의 숫자를 변수로 옮기기

```
# 문제의 숫자들 (단위: 만 원)
SERVER_INIT = 1200 # 서버 초기 구매비
SERVER_MONTH = 20 # 서버 월 관리비
CLOUD_RATE = 0.7 # 클라우드 시간당 요금
USAGE = 100 # 월 사용 시간
MONTHS = 36 # 비교 기간 (개월)
```

setup.py

왜 맨 위에 모아 둘까?

- 시나리오를 바꿀 때 한 곳만 고치면 된다
— 파트 2 민감도 분석의 밑준비
- 대문자 이름은 '바꾸지 않는 설정값'이라는 관례



이론 10강의 표를 그대로 코드로 옮겼을 뿐 — 모델링의 첫걸음은 언제나 '숫자에 이름 붙이기'

for문으로 월별 누적 비용 쌓기

```
cost_curve.py

server, cloud = [], []
s, c = SERVER_INIT, 0    # 시작점이 다르다!

for m in range(1, MONTHS + 1):
    s += SERVER_MONTH
    c += CLOUD_RATE * USAGE
    server.append(s)
    cloud.append(c)

print(server[:3], cloud[:3])
```

실행 결과

```
[1220, 1240, 1260]
[70.0, 140.0, 210.0]
```

읽는 법

서버는 1,200에서 출발해 조금씩, 클라우드는 0에서 출발해 성큼성큼. '누적'이므로 매달 값이 리스트에 하나씩 쌓인다.



확인 포인트: 첫 세 달 출력이 왼쪽과 같으면 성공. 아니면 s, c의 시작값부터 점검!

역전의 순간을 코드로 찾기



crossover.py

```
for m in range(MONTHS):  
    if server[m] <= cloud[m]:  
        print(m + 1, "개월째에 역전!")  
        break    # 처음 한 번만 찾고 종료
```

실행 결과

24 개월째에 역전!

이론과 일치!

$1,200 \div (70 - 20) = 24$ 개월. 공식과 코드가 같은 답을 낸다.

왜 굳이 코드로? 지금은 공식 한 줄로도 풀린다. 하지만 요금이 매년 오르거나, 사용량이 달마다 다르면 공식은 없다 — 코드는 그때도 통한다.

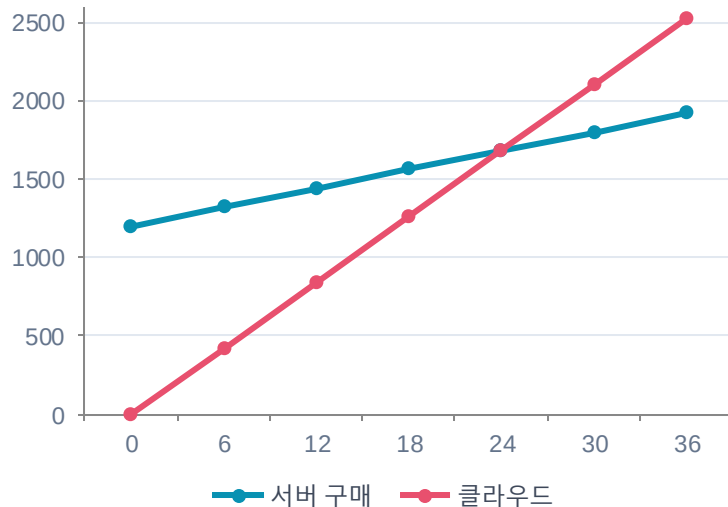
그래프로 그려서 눈으로 확인



plot.py

```
import matplotlib.pyplot as plt

plt.plot(server, label="Buy server")
plt.plot(cloud, label="Cloud")
plt.xlabel("Month")
plt.ylabel("Total cost")
plt.legend()
plt.show()
```



예상 결과 — 24개월에서 교차하는 X자 모양이면 성공



팁: Colab에서 한글 라벨은 깨진다 — 라벨은 영어로 쓰거나, 한글 폰트 설치 방법은 과제 자료 참고.

내 손으로 확인하기 ① (7분)

M1

서버 가격이 1,500만 원으로 오르면 교차점은 몇 개월?

힌트: `SERVER_INIT`만 바꾸고 전체 다시 실행 (예상: 30개월)

M2

월 관리비가 30만 원이라면? 교차점은 어떻게 움직일까?

힌트: 바꾸기 전에 먼저 예측하고, 코드로 확인 (예상: 30개월)

M3

비교 기간을 60개월로 늘려 그래프를 다시 그려 보자

힌트: `MONTHS = 60` — 변수를 위에 모아 둔 보람을 느끼는 순간

다 끝난 사람은: 교차점을 그래프 위에 점으로 표시하는 방법을 검색해 볼 것 (`plt.scatter`)

PART 2

민감도 분석

25분

“만약 ~라면?”을 수십 번 물어도 코드는 지치지 않는다

- 1 반복되는 계산을 함수로 묶기 — def
- 2 사용량 시나리오 5개를 for문으로 한꺼번에
- 3 결과 해석: 결론이 뒤집히는 경계 찾기

반복되는 계산은 함수로 묶는다



sensitivity.py

```
def break_even(init_gap, month_gap):  
    # 초기비 격차를 매달 얼마씩 따라잡나  
    return init_gap / month_gap  
  
print(break_even(1200, 50))
```

실행 결과

24.0

함수 = 이름 붙인 계산기

재료(입력)를 넣으면 답(출력)이 나온다. 한번 정의하면 몇 번이고 재사용.



같은 계산을 두 번 이상 하게 되면 그때가 함수를 만들 타이밍이다.

시나리오 다섯 개를 한꺼번에



scenarios.py

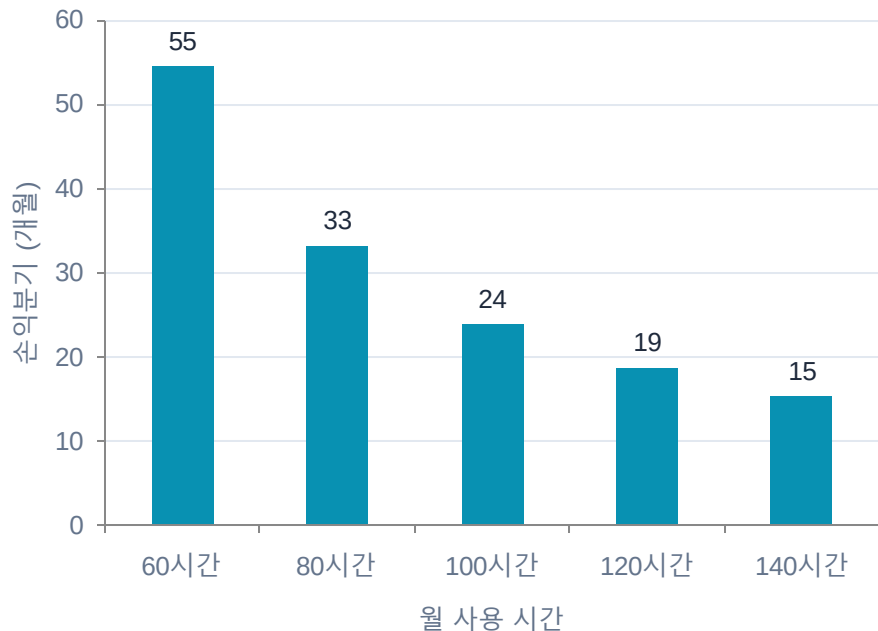
```
for usage in [60, 80, 100, 120, 140]:  
    gap = CLOUD_RATE * usage - SERVER_MONTH  
    be = break_even(1200, gap)  
    print(usage, "시간 →", round(be, 1), "개월")
```

실행 결과

60 시간 → 54.5 개월
80 시간 → 33.3 개월
100 시간 → 24.0 개월
120 시간 → 18.8 개월
140 시간 → 15.4 개월

이것이 민감도 분석. 가정(사용량)을 흔들었더니 답(손익분기)이 15개월에서 55개월까지 움직인다 — 이 답은 가정에 '민감'하다.

가정이 흔들리면 답이 얼마나 움직이나



사용량이 적으면

손익분기 4년 반 — 그 사이 장비는 낡는다. 사실상 클라우드가 답.

3년 안에 본전 조건

월 76시간 이상 써야 36개월 내 회수. 이 '76'이 파트 3의 열쇠가 된다.

내 손으로 확인하기 ② (7분)

M1

클라우드가 요금을 20% 인하하면(0.7 → 0.56) 손익분기는?

예상: 33.3개월 — 요금 인하 하나로 '2년이면 본전'이 '3년 가까이'로

M2

관리비 30만 원 + 사용량 80시간, 두 가정이 동시에 나빠지면?

예상: 46.2개월 — 나쁜 가정은 겹치면 훨씬 아프다

M3

어떤 조건이면 '영원히 클라우드가 유리'해질까?

힌트: 월 격차(gap)가 0 이하가 되는 사용량은? (28.6시간 이하)

다 끝난 사람은: 시나리오 결과를 리스트에 모아 막대그래프로 그려 볼 것 (plt.bar)



10분 휴식

다음 파트는 오늘의 하이라이트 — 10,000개의 미래를 살아 보는 몬테카를로입니다.

PART 3

30분

몬테카를로 시뮬레이션

미래 사용량이 '항상 100시간'일 리 없다 — 9강의 도구를 꺼낼 시간

- 1 random.gauss로 불확실한 사용량 흉내 내기
- 2 미래 하나를 통째로 계산하는 함수 만들기
- 3 10,000번 반복 → '구매가 이득일 확률'
- 4 불확실성의 크기를 바꿔 보기 (심화)

불확실한 미래를 난수로 흉내 내기



future.py

```
import random

for _ in range(5):    # 미래 다섯 개만 미리보기
    print(round(random.gauss(100, 40)))
```

실행 결과

96
152
71
118
43
(실행할 때마다 다름!)

`random.gauss(100, 40)` = “평균은 100시간일 것 같지만, ± 40 시간쯤은 출렁일 수 있다”

어떤 미래에는 43시간밖에 안 쓰고(프로젝트 축소), 어떤 미래에는 152시간을 쓴다(서비스 대박). 우리가 모르는 미래를 확률분포로 표현하는 것 — 9강의 '무작위로 흉내 내기' 그대로다.

미래 하나를 통째로 계산하기



one_future.py

```
def server_total(m):  
    return 1200 + 20 * m  
  
def cloud_total(usage, m):  
    return 0.7 * usage * m  
  
usage = random.gauss(100, 40) # 미래 하나  
print(server_total(36), "vs",  
      round(cloud_total(usage, 36)))
```

실행 결과

1920 vs 2101

이 미래에서는 구매 승!

3년 총비용: 서버 1,920 vs 클라우드 2,101.
하지만 다른 미래에서는? — 한 번으로는
알 수 없다.



여러 번 실행해 보자 — 이길 때도, 질 때도 있다. 그래서 '아주 많이'가 필요하다.

10,000개의 미래를 살아 보기

```
monte_carlo.py

N = 10000
win = 0

for _ in range(N):
    usage = random.gauss(100, 40)
    if server_total(36) < cloud_total(usage, 36):
        win += 1

print("구매가 이득일 확률:", win / N)
```

실행 결과

구매가 이득일 확률: 0.7194

약 72%

9강 몬테카를로 3단계 그대로: 흉내(gauss)
→ 반복(1만 번) → 비율(win/N). 코드 아홉 줄이 전부다.



값이 0.71~0.73 사이면 정상 — 난수라서 매번 조금씩 다르다 (9강: 큰 수의 법칙).

“24개월”이 “72%”가 되기까지

이론의 답 (확실한 세계)

“사용량이 100시간 그대로라면, 24개월 뒤 본전을 뺍습니다.”

가정이 맞을 때만 유효한, 조건부의 답

시뮬레이션의 답 (불확실한 세계)

“3년 기준, 서버 구매가 이득일 확률은 약 72%입니다.”

가정의 불확실성까지 담아낸, 보고서에 쓸 수 있는 답

그리고 28%의 미래에서는 손해를 본다. 그 위험을 감수할지는 계산이 아니라 선택의 문제 — 9강에서 배운 '위험 성향'이 여기서 등장한다.

불확실성이 커지면 확신은 줄어든다

```
uncertainty.py

for sd in [20, 40, 60]:    # 표준편차 3종
    win = 0
    for _ in range(10000):
        u = random.gauss(100, sd)
        if server_total(36) < cloud_total(u, 36):
            win += 1
    print(sd, "→", win / 10000)
```

실행 결과

20 → 0.89
40 → 0.73
60 → 0.66

미래가 흐릿할수록

'지금 목돈을 묶는' 결정의 확신이 떨어진다는 것
→ 불확실할 땐 변동비형(클라우드)이 안전하다.



표준편차는 데이터가 아니라 '분석가의 가정' — 그래서 바꿔 보는 것 자체가 민감도 분석이다.

내 손으로 확인하기 ③ (7분)

M1

N을 100 → 1,000 → 10,000으로 바꿔 확률이 얼마나 출렁이는지 보자

9강 '큰 수의 법칙'의 재확인 — N이 클수록 답이 안정된다

M2

비교 기간이 24개월이라면 확률은? 먼저 예측하고 실행!

예상: 약 50% — 24개월은 딱 손익분기라서 동전 던지기가 된다

M3

팀이 성장해 평균 사용량이 120시간이 되면 확률은?

힌트: `gauss(120, 40)` — 예측 → 실행 → 해석 순서로

다 끝난 사람은: 10,000개 미래의 클라우드 총비용을 히스토그램으로 (`plt.hist`)

이번엔 여러분 차례: 검사 자동화 장비

조별(2~3명)로 15분. 오늘 만든 코드를 복사해 변수만 바꾸면 됩니다 — 처음부터 끝까지, 확률로 답하세요.

항목	값
장비 도입 비용	5,000만 원 (한 번만)
유지비	월 30만 원
인건비 절감(예상)	월 200만 원 — 단, 불확실! gauss(200, 60)
사용 기간	48개월

답해야 할 것

- 절감액이 예상(200만)대로일 때 손익분기는 몇 개월?
- 4년 기준, 도입이 이득일 확률은?
- 당신이 팀장이라면? — 권고안 한 문장



발표 형식: “손익분기 ○개월, 이득일 확률 ○%, 따라서 우리 조는 ○○를 권고합니다.”

발표가 끝난 뒤에 함께 확인



challenge.py

```
# 손익분기: 5000 ÷ (200 - 30) = 29.4개월
N, win = 10000, 0
for _ in range(N):
    save = random.gauss(200, 60) # 불확실
    cost = 5000 + 30 * 48
    if cost < save * 48:
        win += 1
print(win / N)
```

실행 결과

0.8663

권고안 예시

"손익분기 29.4개월, 4년 기준 이득일 확률 약 87% — 도입을 권고합니다. 단, 절감액이 예상보다 30% 낮으면 손해이므로 6개월 후 실적 재검토를 조건으로."



좋은 권고안 = 숫자(확률) + 결론 + 조건. '재검토 조건'까지 붙이면 실무 수준이다.

오늘의 핵심 정리

- ✓ 누적 비용 곡선은 for문과 리스트면 충분하다 — 공식이 없어도 코드는 통한다
- ✓ 함수로 묶으면 "만약 ~라면?" 수십 번이 공짜 — 민감도 분석
- ✓ 몬테카를로: 미래를 10,000번 살아 보고 확률로 답한다 (흉내 → 반복 → 비율)
- ✓ 좋은 보고는 "24개월"이 아니라 "이득일 확률 72%, 단 이런 조건에서"



과제 (다음 시간까지)

내 실생활 선택 하나(정기권 vs 회당 결제, 헬스장 연간권, 배달 구독 등)를 오늘 코드로 분석해 노트북 제출 — 손익분기 + 몬테카를로 확률 + 한 줄 결론 포함.